

WEEK 1 · MODULE 1

Software Design and Architecture

Introduction: what architecture is, and why it costs what it costs

What this course covers

From the principles inside one class to the trade-offs of a whole distributed system, and how to write both down so someone else can trust the decision.

10

weekly modules

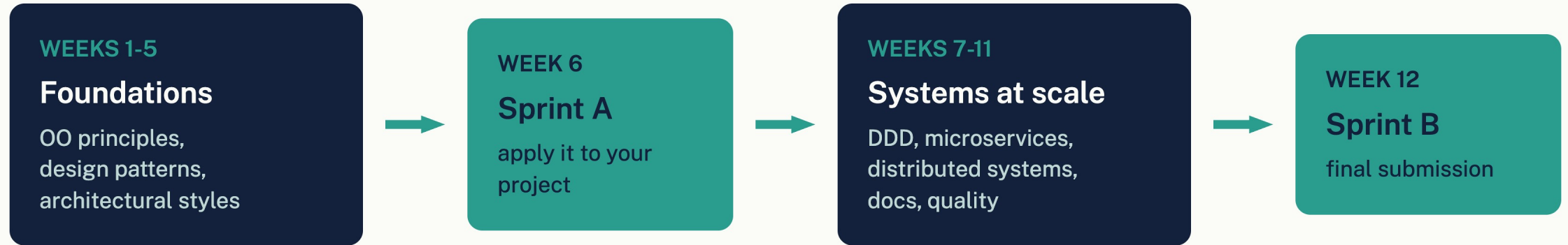
2

project sprints

1

group project, built up all term

The roadmap



Today: Module 1, the first of five foundation modules before Sprint A.

Where to find things

Item	Where
Course site and syllabus	GitHub repo README, top level
This week's module	m1_introduction/README.md
Exercise starter files	each module's exercise_files/ folder
Report template	reports/README.md
Books and further reading	README, "Course books" section

MODULE 1

Learning objectives

- Distinguish software architecture from software design, and know why the distinction matters
- Understand technical debt as a metaphor for the cost of postponing structural decisions
- Recognize the major quality attributes and name the trade-offs between them
- Identify the stakeholders whose concerns shape an architecture, and why they often conflict

Architecture versus design

Design

Decisions inside one component. Cheap to change: usually an afternoon, editing a class.

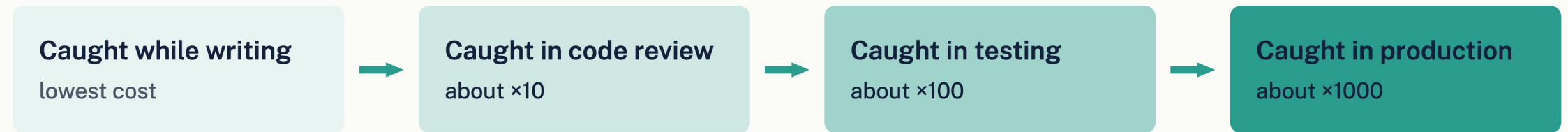
Architecture

Decisions about the system's structure. Expensive to change: you'd need a migration plan.

Swapping a sorting algorithm is design. Swapping a relational database for an event log every service reads from is architecture.

Why it matters: the cost of change

Technical debt: a structural shortcut moves you faster now, at the cost of interest paid on every future change that touches the same area.



Quality attributes: the "-ilities"

Attribute	What it means	A common cost
Performance	How fast the system responds under load	More caching, more moving parts to keep consistent
Availability	How much uptime the system delivers	Redundancy, more operational complexity
Security	Resistance to unauthorized access or misuse	Extra validation and access-control layers
Maintainability	How cheaply the system can be changed later	Time spent on structure now, not features now
Testability	How easily correctness can be verified	Extra seams: interfaces, dependency injection
Scalability	How gracefully the system handles growth	Distributed-systems complexity (Module 8)
Usability	How easily users accomplish their goals	Constraints on internal structure

A TRADE-OFF, MADE CONCRETE

Adding a cache improves performance, at the cost of maintainability

Now two copies of the same data can disagree, and every future change has to reason about both.

Stakeholders rarely agree



Product owner

time-to-market, features



Operations team

availability, observability



Security team

attack surface



Future engineer

you, in six months:
maintainability

The fastest path to a demo often damages maintainability. The most secure design often damages usability.

Project: quality attribute audit

Pick a small, real open-source project on GitHub. Read its directory and module structure, then:

1. Identify 2-3 quality attributes its structure seems to prioritize, with real evidence
2. Identify one quality attribute it seems to have sacrificed, with evidence
3. Write 200-300 words connecting the evidence to your conclusion

Required, due before Module 2.

Choosing a project domain

Pick something with enough real complexity to need a domain model, at least two natural service boundaries, and a decision worth writing an ADR about. Some starting points:

Library management system

Ride-sharing dispatch service

Event ticketing platform

Food ordering and delivery

Hotel booking system

Course registration system

Warehouse inventory and orders

Turn-based multiplayer game backend

Personal finance tracker

Summary

- Architecture and design differ mainly in how expensive a decision is to reverse
- Technical debt accumulates when structural decisions get deferred past the point where they're cheap to make well
- Every architectural decision trades one quality attribute against another
- Stakeholders rarely agree on which attribute matters most, and that has to be made explicit, not assumed

NEXT WEEK

Module 2: Object-Oriented Design Principles

Coupling, cohesion, and the five SOLID principles, each with a real before and after refactor.

Questions?

Ali Gunes · ali.gunes@atlas.edu.tr